

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

GA – Práctica en clase

Proyecto ASP.NET Core MVC

MiPrimerMVC

Asignatura: Aplicaciones Web

Estudiante: Mariscal Cabrera Jaime Josué

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Periodo Académico: 2025–2026 SPA

Repositorio: <https://github.com/Grinjaww/MiPrimerMVC>

Quevedo, Ecuador

18 de junio de 2026

1. Objetivo

Desarrollar una aplicación web sencilla en **ASP.NET Core MVC**, utilizando un controlador de productos con las acciones **Index** y **Create**. En la práctica se considera la creación del modelo **Producto**, la aplicación de reglas de validación mediante *Data Annotations*, el uso de una vista Razor con **AntiForgeryToken** y la comprobación de la validación realizada desde el servidor.

1.1. Requisitos previos

Para realizar la práctica se requiere contar con los siguientes elementos:

- .NET SDK 8.0 o superior instalado.
- Visual Studio 2022 o Visual Studio Code.
- Navegador web para probar la aplicación.

La versión instalada del SDK puede revisarse desde la terminal con el comando `dotnet --version`.

2. Paso 1 – Crear el proyecto MVC

Se inició creando el proyecto base con la plantilla MVC proporcionada por .NET. Para ello, se ejecutaron los siguientes comandos desde la terminal:

```
dotnet new mvc -n MiPrimerMVC
cd MiPrimerMVC
```

Listing 1: Creación del proyecto

Este comando genera la estructura inicial del proyecto, incluyendo carpetas como **Controllers**, **Models**, **Views**, **wwwroot** y el archivo principal **Program.cs**. Esta organización permite separar la lógica de la aplicación, los datos y la interfaz que observa el usuario.

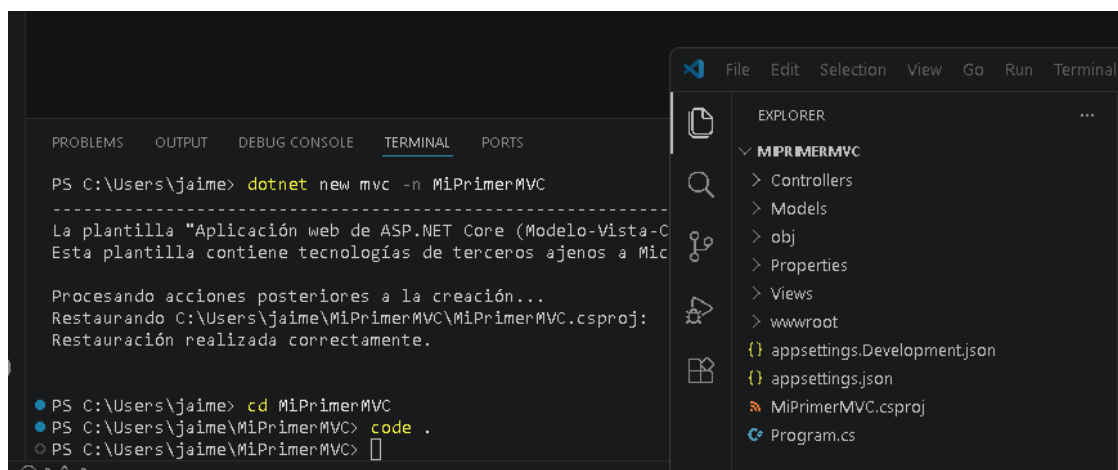


Figura 1: Terminal mostrando la creación del proyecto MiPrimerMVC

3. Paso 2 – Modelo Producto con Data Annotations

Dentro de la carpeta `Models` se creó la clase `Producto`. Esta clase representa los datos que se van a registrar desde el formulario. Además, se agregaron validaciones mediante *Data Annotations*, como campos obligatorios, límites de caracteres, rangos numéricos y formato de correo electrónico.

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace MiPrimerMVC.Models
4 {
5     public class Producto
6     {
7         public int Id { get; set; }
8
9         [Required(ErrorMessage = "El nombre es obligatorio.")]
10        [StringLength(100, MinimumLength = 3,
11            ErrorMessage = "El nombre debe tener entre 3 y 100
12            caracteres.")]
13        [Display(Name = "Nombre del producto")]
14        public string Nombre { get; set; } = string.Empty;
15
16        [Required(ErrorMessage = "La descripción es obligatoria.")]
17        [StringLength(500, ErrorMessage = "La descripción no puede exceder
18            500 caracteres.")]
19        [Display(Name = "Descripción")]
20        public string Descripción { get; set; } = string.Empty;
21
22        [Required(ErrorMessage = "El precio es obligatorio.")]
23        [Range(0.01, 100000, ErrorMessage = "El precio debe estar entre
24            0.01 y 100000.")]
25        [DataType(DataType.Currency)]
26        [Display(Name = "Precio (USD)")]
27        public decimal Precio { get; set; }
28
29        [Required(ErrorMessage = "El stock es obligatorio.")]
30        [Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser
31            negativo.")]
32        [Display(Name = "Stock disponible")]
33        public int Stock { get; set; }
34
35        [EmailAddress(ErrorMessage = "El correo del proveedor no tiene un
36            formato válido.")]
37        [Display(Name = "Correo del proveedor")]
38        public string? CorreoProveedor { get; set; }
39    }
40 }
```

Listing 2: Models/Producto.cs

Con estas anotaciones, ASP.NET Core puede validar automáticamente la información recibida antes de procesarla. Esto ayuda a evitar que se registren datos incompletos o con formatos incorrectos.

```

1  using System.ComponentModel.DataAnnotations;
2
3  namespace MiPrimerMVC.Models
4  {
5      6 references
6      public class Producto
7      {
8          2 references
9          public int Id { get; set; }
10
11          [Required(ErrorMessage = "El nombre es obligatorio.")]
12          [StringLength(100, MinimumLength = 3,
13              ErrorMessage = "El nombre debe tener entre 3 y 100 caracteres.")]
14          [Display(Name = "Nombre del producto")]
15          2 references
16          public string Nombre { get; set; } = string.Empty;
17
18          [Required(ErrorMessage = "La descripcion es obligatoria.")]
19          [StringLength(500, ErrorMessage = "La descripcion no puede exceder 500 caracteres.")]
20          [Display(Name = "Descripcion")]
21          2 references
22          public string Descripcion { get; set; } = string.Empty;
23
24          [Required(ErrorMessage = "El precio es obligatorio.")]
25          [Range(0.01, 100000, ErrorMessage = "El precio debe estar entre 0.01 y 100000.")]
26          [DataType(DataType.Currency)]
27          [Display(Name = "Precio (USD)")]
28          2 references
29          public decimal Precio { get; set; }
30
31          [Required(ErrorMessage = "El stock es obligatorio.")]
32          [Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser negativo.")]
33          [Display(Name = "Stock disponible")]
34          2 references
35          public int Stock { get; set; }
36
37          [EmailAddress(ErrorMessage = "El correo del proveedor no tiene un formato valido.")]
38          [Display(Name = "Correo del proveedor")]
39          2 references
40          public string? CorreoProveedor { get; set; }
41      }
42  }

```

Figura 2: Editor mostrando el archivo Models/Producto.cs

4. Paso 3 – ProductoController con Index y Create

Luego se creó el controlador `ProductoController`. Este controlador contiene la acción `Index`, que muestra el listado de productos, y la acción `Create`, utilizada tanto para abrir el formulario como para recibir la información enviada.

En este caso, los productos se almacenan en una lista en memoria, ya que la práctica no requiere el uso de una base de datos.

```

1  using Microsoft.AspNetCore.Mvc;
2  using MiPrimerMVC.Models;
3
4  namespace MiPrimerMVC.Controllers
5  {
6      public class ProductoController : Controller
7      {
8          // Almacenamiento en memoria. La practica no exige persistencia.
9          private static readonly List<Producto> _productos = new();

```

```
10
11     // GET: /Producto
12     [HttpGet]
13     public IActionResult Index()
14     {
15         return View(_productos);
16     }
17
18     // GET: /Producto/Create
19     [HttpGet]
20     public IActionResult Create()
21     {
22         return View();
23     }
24
25     // POST: /Producto/Create
26     [HttpPost]
27     [ValidateAntiForgeryToken]
28     public IActionResult Create(Producto producto)
29     {
30         // Validacion SERVER-SIDE: si no cumple las Data Annotations,
31         // ModelState.IsValid es false y se regresa la vista con
32         // errores.
33         if (!ModelState.IsValid)
34         {
35             return View(producto);
36         }
37
38         producto.Id = _productos.Count + 1;
39         _productos.Add(producto);
40
41         return RedirectToAction(nameof(Index));
42     }
43 }
```

Listing 3: Controllers/ProductoController.cs

La parte más importante de este controlador se encuentra en el método **Create** de tipo **POST**. Allí se utiliza **ModelState.IsValid** para comprobar si los datos cumplen con las reglas definidas en el modelo. Si existen errores, se devuelve nuevamente el formulario; si todo es correcto, el producto se guarda en la lista y se redirige al listado.

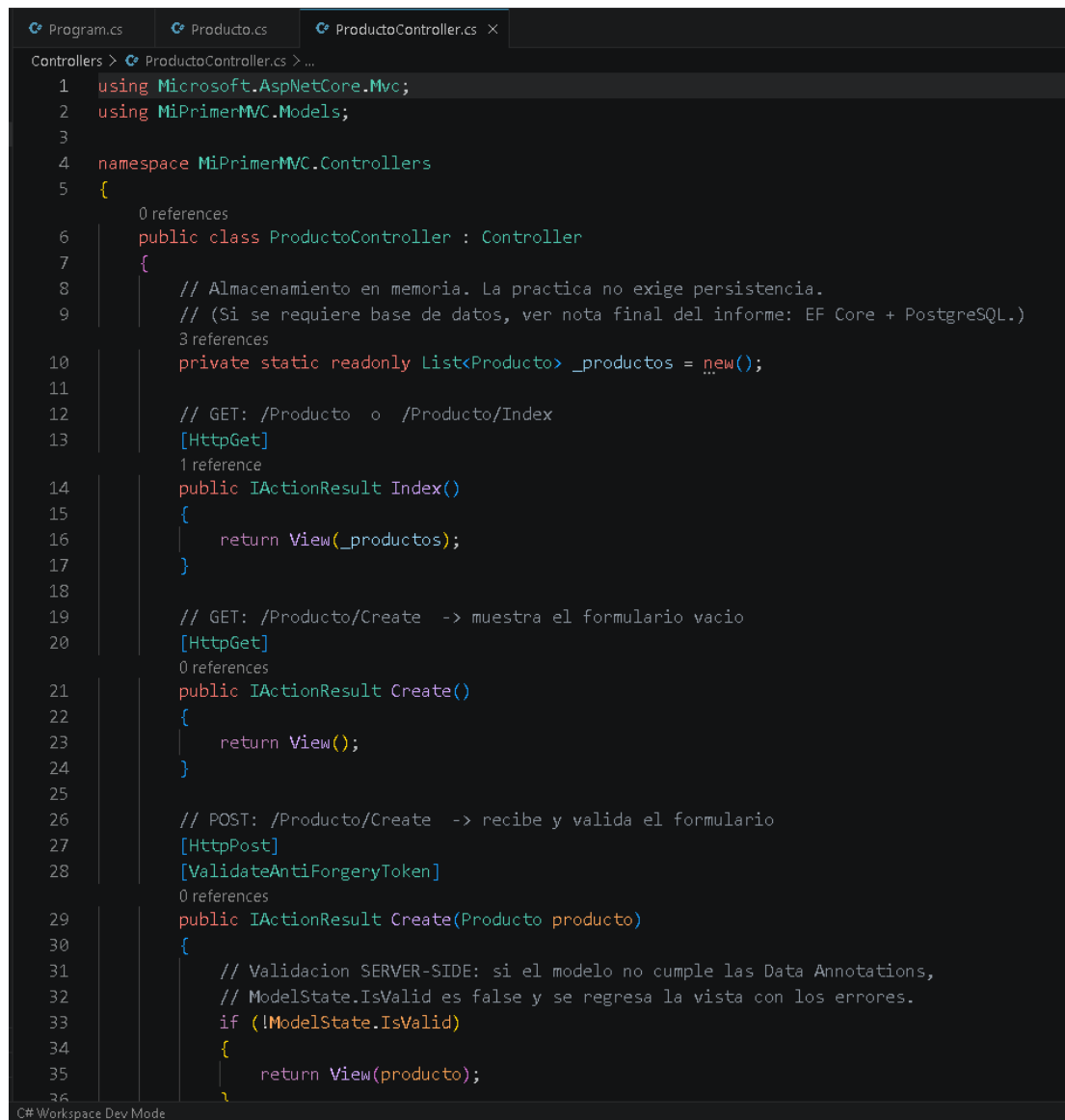


Figura 3: Editor mostrando Controllers/ProductController.cs

5. Paso 4 – Vistas Razor

5.1. Vista Create con AntiForgeryToken

La vista `Create.cshtml` contiene el formulario para registrar un producto. En ella se incluye `@Html.AntiForgeryToken()`, que genera un campo oculto llamado `__RequestVerificationToken`. Este token se valida en el controlador mediante `[ValidateAntiForgeryToken]`.

También se emplean etiquetas auxiliares de Razor, como `asp-for`, `asp-validation-for` y `asp-validation-summary`, para vincular los campos del formulario con el modelo y mostrar los mensajes de error correspondientes.

```

1  @model MiPrimerMVC.Models.Producto
2  @{ ViewData["Title"] = "Crear Producto"; }
3
4  <h1>Crear Producto</h1>

```

```

5 <hr />
6 <div class="row">
7   <div class="col-md-6">
8     /* asp-antiforgery="false" evita el token automatico para mostrar
9     el token unico de forma explicita con @Html.AntiForgeryToken(). */
10    <form asp-action="Create" method="post" asp-antiforgery="false">
11      @Html.AntiForgeryToken()
12
13      <div asp-validation-summary="All" class="text-danger mb-3"></div>
14
15      <div class="mb-3">
16        <label asp-for="Nombre" class="form-label"></label>
17        <input asp-for="Nombre" class="form-control" />
18        <span asp-validation-for="Nombre" class="text-danger"></span>
19      </div>
20
21      <div class="mb-3">
22        <label asp-for="Descripcion" class="form-label"></label>
23        <textarea asp-for="Descripcion" class="form-control"
24          rows="3"></textarea>
25        <span asp-validation-for="Descripcion" class="text-danger"></span>
26      </div>
27
28      <div class="mb-3">
29        <label asp-for="Precio" class="form-label"></label>
30        <input asp-for="Precio" class="form-control" />
31        <span asp-validation-for="Precio" class="text-danger"></span>
32      </div>
33
34      <div class="mb-3">
35        <label asp-for="Stock" class="form-label"></label>
36        <input asp-for="Stock" class="form-control" />
37        <span asp-validation-for="Stock" class="text-danger"></span>
38      </div>
39
40      <div class="mb-3">
41        <label asp-for="CorreoProveedor" class="form-label"></label>
42        <input asp-for="CorreoProveedor" class="form-control" />
43        <span asp-validation-for="CorreoProveedor"
44          class="text-danger"></span>
45      </div>
46
47      <button type="submit" class="btn btn-primary">Guardar</button>
48      <a asp-action="Index" class="btn btn-secondary">Volver al listado</a>
49    </form>
50  </div>
51
52  /* No se incluye _ValidationScriptsPartial: asi la validacion ocurre
53  en el servidor y puede verificarse el round-trip. */

```

Listing 4: Views/Producto/Create.cshtml

5.2. Vista Index

La vista `Index.cshtml` muestra los productos registrados. Si todavía no existe ningún producto, se presenta un mensaje informativo. Cuando ya hay registros, la información se organiza en una

tabla.

```
1  @model IEnumerable<MiPrimerMVC.Models.Producto>
2  @{ ViewData["Title"] = "Listado de Productos"; }
3
4  <h1>Productos</h1>
5  <p><a asp-action="Create" class="btn btn-primary">Crear nuevo
    producto</a></p>
6
7  @if (!Model.Any())
8  {
9      <div class="alert alert-info">Aun no hay productos registrados.</div>
10 }
11 else
12 {
13     <table class="table table-striped">
14         <thead>
15             <tr><th>Id</th><th>Nombre</th><th>Descripcion</th>
16                 <th>Precio</th><th>Stock</th><th>Correo</th></tr>
17         </thead>
18         <tbody>
19             @foreach (var item in Model)
20             {
21                 <tr>
22                     <td>@item.Id</td><td>@item.Nombre</td>
23                     <td>@item.Descripcion</td><td>@item.Precio.ToString("C")</td>
24                     <td>@item.Stock</td><td>@item.CorreoProveedor</td>
25                 </tr>
26             }
27         </tbody>
28     </table>
29 }
```

Listing 5: Views/Producto/Index.cshtml


```

views > Producto > Index.cshtml
1  @model IEnumerable<MiPrimerMVC.Models.Producto>
2
3  @{
4      ViewData[index: "Title"] = "Listado de Productos";
5  }
6
7  <h1>Productos</h1>
8
9  <p>
10     <a asp-action="Create" class="btn btn-primary">Crear nuevo producto</a>
11 </p>
12
13 @if (!Model.Any())
14 {
15     <div class="alert alert-info">Aun no hay productos registrados.</div>
16 }
17 else
18 {
19     <table class="table table-striped">
20     <thead>
21     <tr>
22         <th>Id</th>

```

```

views > Producto > Create.cshtml
1  @model MiPrimerMVC.Models.Producto
2
3  @{
4      ViewData[index: "Title"] = "Crear Producto";
5  }
6
7  <h1>Crear Producto</h1>
8  <hr />
9
10 <div class="row">
11     <div class="col-md-6">
12         @* asp-antiforgery="false" desactiva el token automatico del tag helper
13            para mostrar el token UNICO de forma explicita con @Html.AntiForgeryToken(). *@
14         <form asp-action="Create" method="post" asp-antiforgery="false">
15             @Html.AntiForgeryToken()
16
17             @* Resumen de errores de validacion del lado del servidor *@
18             <div asp-validation-summary="All" class="text-danger mb-3"></div>
19
20             <div class="mb-3">
21                 <label asp-for="Nombre" class="form-label"></label>

```

Figura 4: Editor mostrando las vistas Create e Index

6. Paso 5 – Configuración de la ruta

En el archivo `Program.cs` se modificó la ruta por defecto para que la aplicación inicie directamente en el controlador de productos, específicamente en la acción `Index`.

```

1  app.MapControllerRoute(
2      name: "default",
3      pattern: "{controller=Producto}/{action=Index}/{id?}");

```

Listing 6: Program.cs

Con esta configuración, al ejecutar el proyecto se accede primero al listado de productos.

7. Ejecución y resultados

Para ejecutar la aplicación se utilizó el siguiente comando:

```
dotnet run
```

Listing 7: Ejecución del proyecto

Una vez iniciado el servidor, se abrió la aplicación desde el navegador para comprobar el funcionamiento de las vistas y del formulario.

7.1. Listado inicial

Al ingresar al listado por primera vez, la aplicación muestra un mensaje indicando que todavía no hay productos registrados.

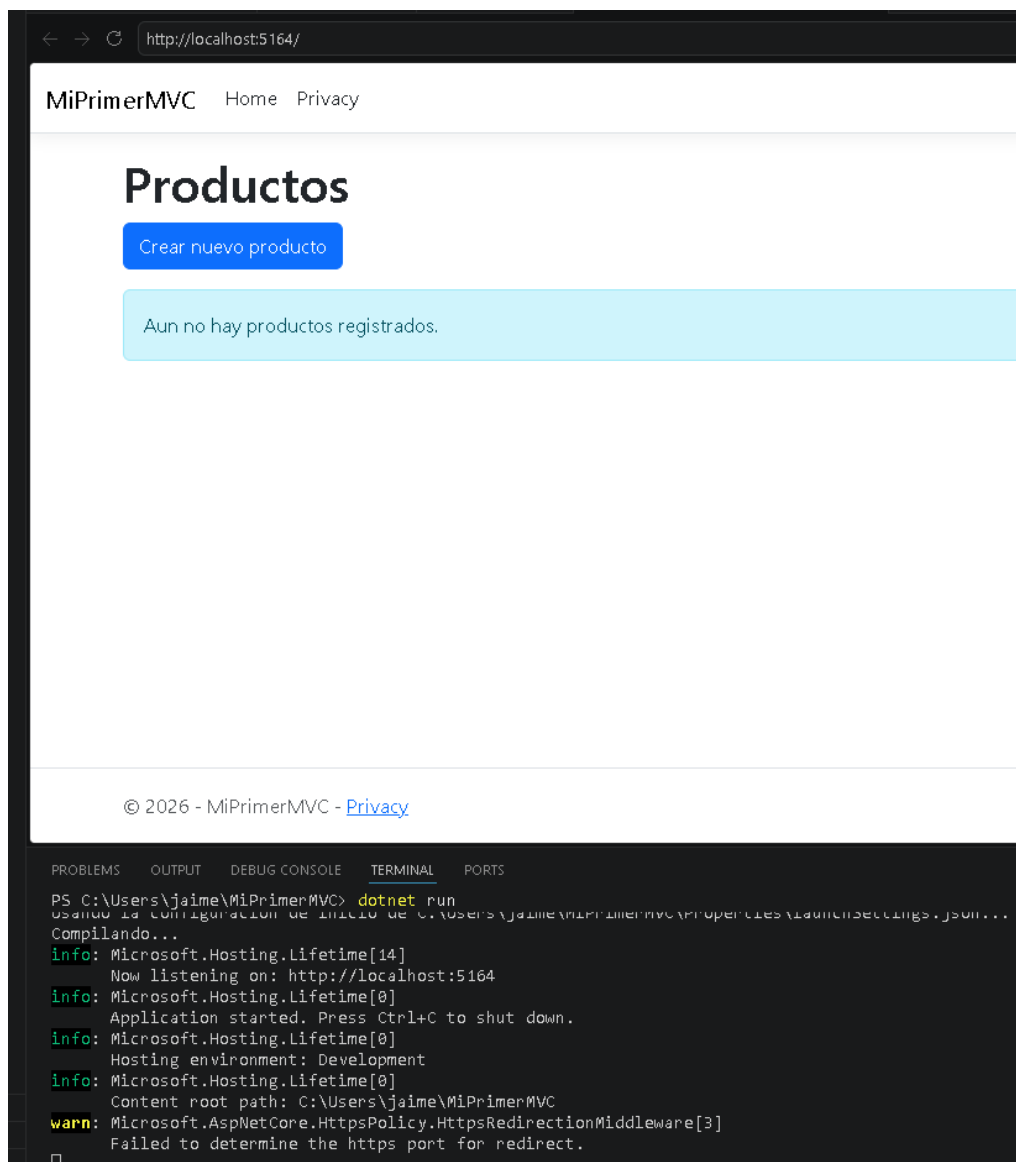


Figura 5: Navegador en /Producto/Index con la lista vacía

7.2. Formulario de creación

Desde el listado se accede al formulario de creación, donde se ingresan los datos del producto.

The screenshot shows a web browser at the URL `http://localhost:5164/Producto/Create`. The page title is 'MiPrimerMVC' with links for 'Home' and 'Privacy'. The main heading is 'Crear Producto'. Below it are five input fields: 'Nombre del producto' (text), 'Descripcion' (text area), 'Precio (USD)' (text), 'Stock disponible' (text), and 'Correo del proveedor' (text). At the bottom of the form are two buttons: 'Guardar' (blue) and 'Volver al listado' (grey). The footer contains the text '© 2026 - MiPrimerMVC - [Privacy](#)'.

Figura 6: Navegador en `/Producto/Create` mostrando el formulario

7.3. Token AntiForgery en el HTML generado

Al revisar el código fuente de la página, se observa el campo oculto generado por `@Html.AntiForgeryToken()`:

```
<input name="__RequestVerificationToken" type="hidden" value="CfDJ8N..." />
```

Listing 8: Campo oculto generado por `AntiForgeryToken`

Este campo permite que el servidor verifique que el formulario enviado pertenece a la aplicación y no proviene de una solicitud externa no autorizada.

```

<!-- Crear Producto />
<hr />

<div class="row">
  <div class="col-md-6">
    <form method="post" action="/Producto/Crear">
      <input name="requestVerificationToken" type="hidden" value="CFD810rZ4uF_Z51uVH54FzHf2G20-zxrN6W3jntDm7mYnDm9_kVLaR6uZW6H9W22R6HwtuFA1021UK46p7zCR6g7Jm9ryqELTMy9S7eeXmGdm0t_0m9Qm3S5-15duw5QyKufUPAF7_BwLmLm />
      <div class="text-danger mb-3 validation-summary-valid" data-valmsg-summary="true"><div style="display:none"></div></div>
      <div class="mb-3">
        <label class="form-label" for="Nombre">Nombre del producto</label>
        <input class="form-control" type="text" data-val="true" data-val-length="El nombre debe tener entre 3 y 100 caracteres." data-val-length-max="100" data-val-length-min="3" data-val-required="El nombre es obligatorio." />
        <span class="text-danger field-validation-valid" data-valmsg-for="Nombre" data-valmsg-replace="true"></span>
      </div>
    </form>
  </div>
</div>

```

Figura 7: Código fuente del navegador mostrando 'RequestVerificationToken'

8. Verificación de la validación server-side

Para comprobar que la validación se realiza desde el servidor, se enviaron datos incorrectos desde el formulario. Por ejemplo, campos vacíos, un precio igual a 0 o un correo con formato inválido.

Como no se incluyeron los scripts de validación del cliente, el formulario se envía al servidor. Luego, el controlador evalúa `ModelState.IsValid`. Cuando el resultado es `false`, la aplicación devuelve la misma vista con los mensajes de error definidos en el modelo.

8.1. Caso 1: formulario vacío

Crear Producto

- El nombre es obligatorio.
- La descripción es obligatoria.
- The value " " is invalid.
- The value " " is invalid.

Nombre del producto

El nombre es obligatorio.

Descripción

La descripción es obligatoria.

Precio (USD)

The value " " is invalid.

Stock disponible

The value " " is invalid.

Correo del proveedor

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 8: Formulario enviado vacío mostrando mensajes de error

8.2. Caso 2: datos inválidos

En este caso se prueba el formulario con un correo electrónico no permitido.

Correo del proveedor

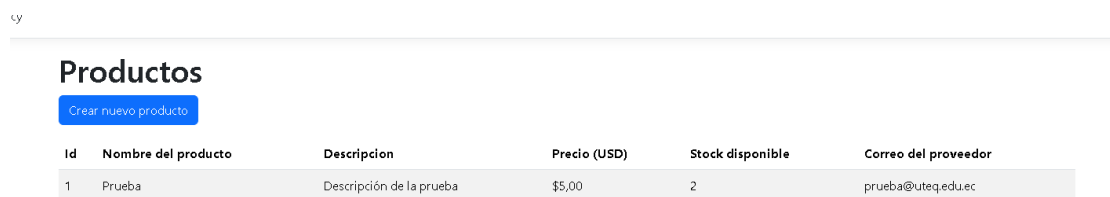
 Incluye un signo "@" en la dirección de correo electrónico. La dirección "abc" no incluye el signo "@".

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 9: Formulario con precio 0 y correo inválido

8.3. Caso 3: datos válidos

Cuando la información ingresada cumple con las reglas establecidas, el producto se registra correctamente y la aplicación redirige al listado.



The screenshot shows a web page titled 'Productos'. Below the title is a blue button labeled 'Crear nuevo producto'. Below the button is a table with the following data:

Id	Nombre del producto	Descripción	Precio (USD)	Stock disponible	Correo del proveedor
1	Prueba	Descripción de la prueba	\$5,00	2	prueba@uteq.edu.ec

Figura 10: Listado mostrando el producto registrado

8.4. Evidencia opcional: punto de interrupción

Como comprobación adicional, se puede colocar un *breakpoint* en la condición `if (!ModelState.IsValid)` dentro del método `Create` de tipo `POST`. Esto permite observar que la validación pasa por el servidor antes de aceptar o rechazar los datos.

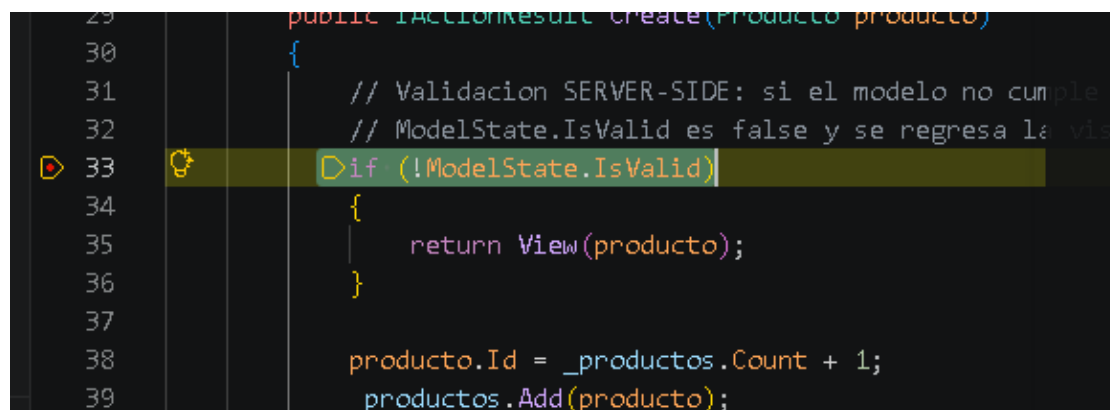


Figura 11: Depurador detenido en `ModelState.IsValid`

9. Conclusiones

- Se desarrolló un proyecto ASP.NET Core MVC funcional, utilizando un controlador de productos con las acciones principales `Index` y `Create`.
- El modelo `Producto` permitió centralizar las reglas de validación mediante *Data Annotations*, lo que facilita controlar los datos antes de procesarlos.
- La vista Razor incluyó **AntiForgeryToken**, que junto con `[ValidateAntiForgeryToken]` añade una capa de seguridad al formulario.
- Se comprobó que la validación server-side se ejecuta correctamente mediante `ModelState.IsValid`, sin depender únicamente de validaciones del navegador.